

Best Practice for AI Agents Project

Chapter 3

Injecting Private Capabilities with Skills, Tools, and MCP

Junxian Zhu¹ Yiran Sun² Guanping Dai³

LinkMind Project

¹ Junxian Zhu, LinkMind Project Team, [LandingBJ](#)

² Yiran Sun, LinkMind Project Team, [Xiamen University Malaysia](#)

³ Guanping Dai, LinkMind Project Team, Founder and CEO of [LandingBJ](#)

Contents

About the Contributors	2
Chapter 3. Injecting Private Capabilities with Skills, Tools, and MCP	3
Learning Objectives	4
3.1 What a Capability Actually Is	4
3.2 Read Actions, Write Actions, and the Ethics of Side Effects	5
3.3 Skills, Tools, and MCP as Capability Forms	5
3.4 Designing the Capability Contract	6
3.5 Hands-On Lab: A Shared Capability Layer for Asset Support . . .	7
3.6 Capability Testing and Release Criteria	10
3.7 Common Anti-Patterns	10
3.8 Human Approval and Reversible Workflow Design	10
3.9 Capability Versioning and Portfolio Management	11
3.10 Sandboxing, Quotas, and Rate Governance	12
3.11 Capability Readiness Checklist	13
Chapter Summary	13
Review Questions	13
Further Study: The Capability Lifecycle from Prototype to Platform Asset	14
Further Study: Economic Design and Portfolio Discipline	15

About the Contributors

Junxian Zhu

LinkMind Project Team, [LandingBJ](#)

- Focus on the research and development of large model products, covering algorithms and applications
- Rich experience in full-process development and delivery of government and enterprise projects

Yiran Sun

LinkMind Project Team, [Xiamen University Malaysia](#)

- Focus on AI agent engineering practices and applied AI workflows
- Background in computer science, machine learning, and AI-related system research

Guanping Dai

LinkMind Project Team, Founder and CEO of [LandingBJ](#)

- Over 20 years of experience in AI algorithm research and development. Built proprietary deep learning and large-model frameworks
- Extensive experience in banking and telecommunications system architecture. Participated in image analysis projects for the National Automotive Safety Laboratory and telemetry tracking systems for Shenzhou spacecraft recovery capsules
- Former experience at the Chinese Academy of Sciences Software Institute
- Former experience at BEA/Oracle (China)

3

CHAPTER

Injecting Private Capabilities with Skills, Tools, and MCP

Knowledge makes agents informative. Capabilities make them consequential. The distinction matters because it is the moment when agent engineering stops being mostly about answer quality and becomes equally about side effects, permissions, reversibility, and accountability. An assistant that cites a policy incorrectly may mislead a user. An assistant that opens tickets, resets credentials, files procurement requests, or pushes workflow state incorrectly can disrupt operations directly.

Capability injection is therefore not the trivial twin of knowledge injection. It is a different engineering problem. The platform must describe what the agent may do, under which identity, with which parameters, through which integration boundary, and under what review conditions. The design must also represent failure explicitly. A knowledge answer can be uncertain and still useful. A side-effecting action with ambiguous semantics is simply dangerous.

LinkMind's emphasis on preloaded Skills, reusable Tools, and standards-oriented capability integration such as MCP is valuable because it turns capabilities into shared infrastructure rather than local improvisation. If every agent surface defines its own ticketing call, its own calendar semantics, and its own approval assumptions, then reuse collapses and safety becomes inconsistent. A disciplined capability layer is one of the clearest signs that an organization has moved from prototypes to systems.

Learning Objectives

- Distinguish between private knowledge and private capabilities in agent architecture.
- Model a capability as a governed contract rather than a bare function name.
- Understand the roles of Skills, Tools, and MCP in a LinkMind-centered system.
- Design read and write actions with different levels of caution and control.
- Build a shared enterprise capability layer that can be used from Lobster, Hermes Agent, or custom clients without duplicating semantics.

3.1 What a Capability Actually Is

A capability is often presented to language models as a function signature, but an enterprise capability is more than a signature. It has purpose, input schema, side effects, identity assumptions, failure modes, and audit requirements. A function name such as `create_service_ticket` may look self-explanatory to a developer, yet several operational questions remain. Who is allowed to open a ticket? Must the ticket be tied to the authenticated user or can it be opened on behalf of another person? Is priority free-form or enumerated? Can the action be repeated without duplication? What must be logged after the call?

These questions reveal why capability design belongs alongside systems design rather than inside prompting folklore. If the runtime cannot answer them clearly, the agent is forced to infer behavior from incomplete descriptions. That is a fragile place to put operational responsibility. A strong capability layer makes business semantics explicit before the model ever begins to deliberate.

The reader should notice how this differs from classical software integration. In ordinary applications, humans write imperative code that calls a function for a known reason. In agent systems, a model may decide whether the function should be called at all. That makes the contract even more important. The contract is no longer just a convenience for developers; it is a behavioral boundary for the agent.

3.2 Read Actions, Write Actions, and the Ethics of Side Effects

Read-oriented capabilities such as `lookup_asset_owner`, `search_contract_terms`, or `get_ticket_status` widen what the agent knows. Write-oriented capabilities such as `create_service_ticket`, `schedule_meeting`, `initiate_refund`, or `submit_purchase_request` widen what the agent can change. The distinction may seem obvious, but it has deep design consequences. Read operations still require access control, but they rarely require rollback. Write operations can create durable state, trigger workflows, notify humans, or consume money. They therefore demand higher standards of validation and often approval.

Mature agent systems should make this difference visible in the tool registry itself. The platform should know whether a capability is read-only, state-changing, irreversible, or approval-gated. If those properties are hidden inside documentation that the runtime cannot use, then the system cannot protect itself consistently. This is one of the places where a middle layer such as Link-Mind becomes more than an adapter. It becomes a policy-aware capability substrate.

There is also an ethical dimension. As soon as a language model gains the ability to change operational systems, the human expectation of accountability shifts. Users reasonably ask who authorized the action, whether it can be reversed, and what evidence led to it. In this sense, write capabilities are not merely technical endpoints. They are institutional commitments.

3.3 Skills, Tools, and MCP as Capability Forms

Mechanism	Typical Shape	Best Use	Main Governance Question
Tool	A callable operation with a schema	Narrow actions or lookups	Are semantics and permissions explicit?
Skill	A reusable, often composite capability asset	Shared enterprise behaviors reused across agents	How is the asset versioned and approved?
MCP integration	A standardized bridge to external tool ecosystems	Interoperability across broader systems	Which external capabilities are trusted and isolated?

The vocabulary differs across platforms, but the distinctions are useful. A tool is usually the smallest callable unit. A skill often packages a richer behavior, policy, or integration pattern that several agents can reuse. MCP, meanwhile, matters because enterprises increasingly want a common method for exposing external capabilities without writing bespoke glue every time. A textbook treatment should not fetishize the protocol itself; it should ask what problem the protocol solves. The answer is that standardization reduces duplicated integration work while making trust boundaries easier to describe.

LinkMind’s capability story is valuable because it centers reuse. In the platform’s own framing, many skills and tools are meant to be preloaded and shared rather than reinvented for each agent. This is the correct instinct. Reuse is not just about faster development. It is about narrowing the number of places where capability semantics can drift.

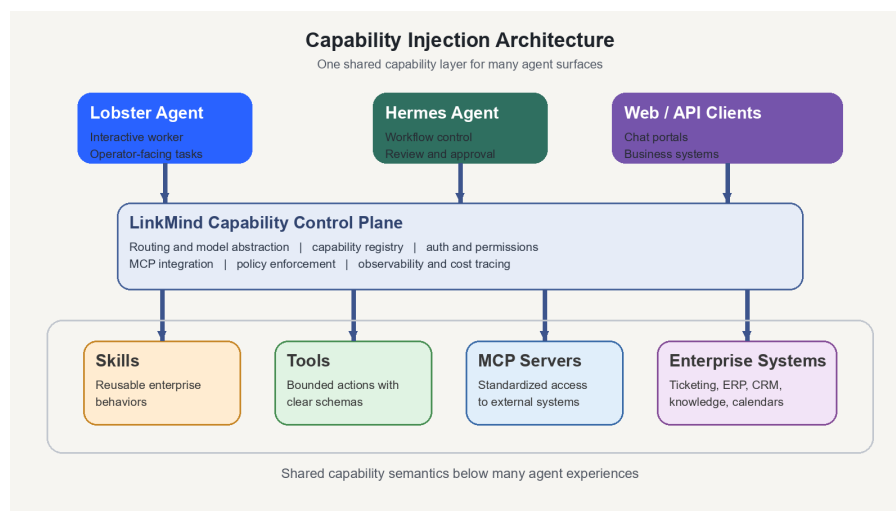


Figure 3.1. Capability injection architecture: agent surfaces, LinkMind control, shared tools, MCP servers, and enterprise systems.

3.4 Designing the Capability Contract

A robust capability contract answers at least seven questions. What business purpose does the action serve? What inputs are required and what types do they accept? Which fields are optional and why? Does the action change state, and if so is it idempotent? Under which user identity or service identity is it executed? What does success look like in a compact observation form? What does failure look like, and how should the agent react? If any one of

these questions is left vague, the model may still call the capability, but the system as a whole becomes harder to trust.

Idempotence deserves special attention. Suppose the user repeats a request because the interface lagged, or the agent retries after an ambiguous timeout. If the capability creates a duplicate ticket, duplicate order, or duplicate notification, then the problem is not only in the model. It is in the contract design. Enterprise capabilities should either be idempotent by construction or expose enough correlation information that duplicates can be detected safely.

The output side of the contract also matters. The observation returned to the model should preserve identifiers, statuses, and next-step affordances while discarding irrelevant backend noise. A ticket creation call might return the ticket ID, queue, priority, and approval status. It should not dump every internal field of the ticketing system unless the downstream task truly requires them.

3.5 Hands-On Lab: A Shared Capability Layer for Asset Support

This lab extends the earlier policy assistant by giving it two enterprise capabilities. The first is a read capability, `lookup_asset_owner`, which returns who owns a device or software asset and which department it belongs to. The second is a write capability, `create_service_ticket`, which opens an internal IT support ticket. These two actions make an excellent teaching pair because they reveal both the convenience and the risk of capability injection.

The lab is also designed to show why centralized capability registration matters. The same capabilities should be usable from a LinkMind-hosted chat, from Lobster as an operator tool, and from Hermes Agent inside a more structured workflow. The user experience may differ across those surfaces. The underlying contract should not.

▷ Step 1. Describe the tool in business language first.

Before writing code or schemas, write one paragraph in plain English for each capability. The paragraph should state the action's purpose, expected inputs, side-effect level, and any approval or identity assumptions. This exercise prevents a common failure mode in which the team mistakes a technical call for a well-modeled business action. Once the prose is clear, translate it into a formal schema.


```
{
  "name": "create_service_ticket",
  "description": "Open an internal IT support ticket for an
  ↪ authenticated employee.",
  "input": {
    "employee_id": "string",
    "asset_id": "string",
    "issue_summary": "string",
    "priority": "low|medium|high"
  },
  "side_effect": true,
  "approval_required": true,
  "idempotency_key": "recommended"
}
```

▷ **Step 2. Register the capability once in LinkMind and consume it from multiple surfaces.**

Expose both capabilities through LinkMind’s shared tool or skill layer. Then verify that a direct LinkMind client, Lobster, and a Hermes Agent workflow can all access the same semantics. The point is not to show that all interfaces look identical. The point is to show that the underlying action meaning, permission behavior, and observation structure remain stable because they are controlled centrally.

This step often produces an organizational insight. Teams that previously believed they each needed separate integration code discover that much of their difference was in interface preference, not in business semantics. A shared capability layer allows the interfaces to diverge where they should and converge where they must.

▷ **Step 3. Force the system to choose between read and write.**

- Prompt A: “Who owns asset LT-2048?” The correct behavior is a read-only lookup.
- Prompt B: “My laptop asset LT-2048 keeps crashing. Open a high-priority ticket and include the summary from our conversation.” The correct behavior is a ticket-creation request with explicit identity and approval logic.

- Prompt C: “Open a ticket for my colleague using their employee ID.” The system should reveal whether delegated action is allowed and under which policy conditions.

These prompts reveal whether the tool descriptions and permissions are sharp enough. If the agent cannot distinguish between merely needing information and initiating a side effect, then the capability boundary is underspecified. If it performs a write without identity context, the design is unsafe even if the call technically succeeds.

▷ **Step 4. Shape observations for downstream reasoning.**

After `lookup_asset_owner`, the model should receive a compact observation such as owner name, department, and asset status. After `create_service_ticket`, it should receive ticket number, queue, priority, created timestamp, and approval status. The observation should make it easy for the model to explain what happened or what remains to be done. This approach reduces token waste and preserves task relevance.

Observation shaping is where a capability layer quietly influences answer quality. Developers sometimes assume that once the action succeeds, the agent can obviously incorporate the result. In practice, the result must still be curated for reasoning. Otherwise the model is handed an object dump and asked to make sense of what the system should already have normalized.

▷ **Step 5. Add one external capability through MCP and inspect the trust boundary.**

Introduce an external capability through an MCP-backed integration, such as a calendar query or a document search endpoint. The purpose is not to demonstrate protocol novelty. It is to force the team to ask which external capabilities deserve the same trust as native ones. Use allow-lists, isolate the exposed actions, and verify that LinkMind still sees enough metadata to route, log, and constrain the capability correctly.

Standardization is useful only when accompanied by governance. Otherwise a standardized bridge merely makes it easier to import poorly understood actions at scale.

3.6 Capability Testing and Release Criteria

Release Question	Why It Matters
Can the agent explain why it chose the capability?	Supports debugging and trust
Are read and write capabilities clearly distinguished in policy and trace?	Prevents silent escalation of risk
Is the action idempotent or duplication-resistant?	Protects operational systems from retries and ambiguity
Can the same capability be consumed consistently from several agent surfaces?	Validates true reuse
Can the platform disable or version the capability centrally?	Supports safe evolution over time

Release criteria for capabilities should be more demanding than those for pure chat behavior. Once enterprise side effects are available, the system is closer to a workflow engine than to a conversational demo. It deserves corresponding rigor in testing, staging, and rollback planning.

3.7 Common Anti-Patterns

- Treating a function signature as a complete capability contract.
- Giving write capabilities to the system before role, identity, and approval are designed.
- Duplicating the same integration semantics in every agent client.
- Ignoring idempotence and then discovering duplicates during retries.
- Importing external capabilities through MCP without a clear trust and isolation policy.

3.8 Human Approval and Reversible Workflow Design

Capability-rich agents are most trustworthy when they understand that some actions require social confirmation as well as technical permission. Approval

is not just a gate. It is a way of expressing institutional judgment. Some actions are low-risk enough to be automated once identity is known. Others require review because they create financial commitments, affect another employee, or touch privileged systems. The system should therefore model approval not as an embarrassing delay in automation, but as a normal stage in enterprise action.

Reversibility matters for a similar reason. If a capability cannot be fully reversed, then the threshold for autonomous use should rise. If it can be reversed, the system should know how that reversal is represented. For example, a created service ticket may be closed or canceled; a procurement request may be withdrawn; a calendar event may be removed. Encoding these possibilities makes the capability layer more realistic and gives human operators confidence that the system does not move only in one direction.

Workflow runtimes such as Hermes Agent are often helpful in expressing approval branches, but the approval semantics themselves should still be shared. If every workflow defines “approval” differently, the organization loses consistency. LinkMind’s role is to keep the meaning of the capability and the visibility of the action stable even when different flows wrap it differently.

Designing for approval and reversibility also improves user experience. Users tolerate automation more readily when they know the system can pause, ask, and undo with clarity rather than acting with irreversible confidence.

3.9 Capability Versioning and Portfolio Management

Over time, a successful platform accumulates many capabilities. Some are stable and widely used. Some are experimental. Some become obsolete but linger because no one owns retirement. Without versioning and portfolio management, the capability surface becomes harder for both models and humans to reason about. A crowded toolset increases the chance of wrong selection, semantic overlap, and governance drift.

Versioning should therefore be explicit. A capability can be superseded while still available for a deprecation window. A new schema can coexist with an older schema while clients migrate. Tool descriptions can indicate preferred versions. Platform teams should also track usage intensity, failure rate, and ownership for each capability. This transforms the capability registry from a pile of integrations into a managed product portfolio.

The portfolio view is particularly useful in LinkMind-centered deployments because many upper-layer agents may depend on the same asset. A poor capability should not remain alive merely because one forgotten workflow still references it. Conversely, a heavily used capability deserves stronger documentation, better evaluation, and clearer release policy than an experimental one.

Mature capability programs often review their registry the way API programs review public interfaces. Which actions are duplicated? Which ones are underused? Which ones should be split or merged? Which ones should never have been agent-accessible in the first place? Such questions prevent the capability layer from decaying into accidental complexity.

3.10 Sandboxing, Quotas, and Rate Governance

Safety is not the only reason to constrain capabilities. Resource fairness and cost governance matter as well. A search capability may be safe but expensive. A document conversion pipeline may be harmless but computationally heavy. A ticketing interface may be correct semantically yet vulnerable to spam-like overuse if an agent loops under poor instructions. Platform teams should therefore think in terms of quotas, rate limits, and sandboxed execution zones even for capabilities that are conceptually benign.

Sandboxing is especially relevant for tool ecosystems exposed through shared runtime layers. Some capabilities should run in isolated execution contexts with strict network and filesystem permissions. Others may require outbound network access but still need strong audit. The point is to prevent the capability layer from becoming a universal escape hatch from the platform's own safety model.

Quotas and rate governance also create useful observability. If a particular agent role is consuming a capability far beyond expectation, the team has learned something about either the workflow or the tool description. LinkMind's promise of centralized control becomes practical here because limits can be applied consistently below many clients instead of being tuned independently and inconsistently at the edges.

From a textbook perspective, this is another reminder that capability engineering and operations engineering are inseparable. A capability is only as good as the environment in which it can be invoked safely, fairly, and intelligibly.

3.11 Capability Readiness Checklist

- ☒ Is the business purpose of the capability written in language that a reviewer outside the implementation team can understand?
- ☒ Are required inputs, optional inputs, and side effects explicit?
- ☒ Does the platform know whether the action is read-only, reversible, approval-gated, or irreversible?
- ☒ Is idempotence handled either directly or through correlation keys and duplicate detection?
- ☒ Can the capability be versioned, disabled, or migrated centrally without breaking every agent surface?
- ☒ Are quotas, logging, and failure observations defined before production rollout?

Chapter Summary

- Private capabilities are governed business actions, not just callable functions.
- Read and write operations must be modeled differently from the start.
- Skills, Tools, and MCP are useful only when their semantics, permissions, and traces are centrally controlled.
- LinkMind is strongest here when it acts as a shared capability substrate beneath several agent surfaces.
- A good capability layer improves reuse, safety, and evaluation simultaneously.

Review Questions

- Why is a side-effecting capability not adequately described by a function name alone?
- How does idempotence change the reliability of agent-driven actions?
- What is the practical difference between a tool, a skill, and an MCP-based capability in an enterprise design?

- Why should several agent surfaces share one capability layer rather than each maintaining its own?
- How would you decide whether an external capability is safe enough to expose to agents?

Further Study: The Capability Lifecycle from Prototype to Platform Asset

Capabilities evolve through stages. At first, they are often rough experiments: a mocked ticketing function, a simple search tool, a manually tested workflow action. If the experiment proves useful, the capability begins to attract more users and more contexts. Identity becomes stricter. Error handling grows more complex. Documentation becomes necessary. Audit fields are requested. Finally, if the capability is genuinely valuable, it becomes a platform asset shared across several agents and business scenarios.

Each stage changes what good engineering looks like. A prototype values speed and clarity of behavioral contract. An adopted capability values schema stability and observability. A platform asset values versioning, ownership, and safe deprecation. Teams frequently get into trouble because they keep treating a later-stage capability as though it were still an early experiment. The system then accumulates unspoken assumptions until the first serious incident or migration forces painful reconstruction.

Thinking in lifecycle terms helps prioritize investment. A high-usage capability deserves better tests, cleaner documentation, and stronger governance than an experimental one. It also helps teams resist premature hardening. Not every experiment needs the full ceremony of a mature platform asset on day one. Good engineering means hardening at the right time, not at the earliest possible moment.

Shared control through LinkMind is useful throughout this lifecycle because it allows the capability's behavioral contract to remain stable even as backend seriousness increases. The platform absorbs part of the maturation burden, sparing each client from having to rediscover it alone.

Further Study: Economic Design and Portfolio Discipline

A capability layer is also a portfolio of investments. Every exposed action carries maintenance cost, governance cost, and cognitive cost for both humans and models. The more crowded the registry becomes, the harder it is for agents to choose well and for operators to govern confidently. Portfolio discipline therefore asks not only whether a capability can be built, but whether it should remain available, to whom, and at what level of support.

The best portfolio decisions usually favor high-leverage shared actions. A capability that shortens work across three departments, reduces error, and can be governed cleanly is a strong candidate for platform elevation. A capability that mainly exists because it looks impressive in a product demo may not justify long-term cost. Textbooks on agent engineering should teach this explicitly, because otherwise teams are tempted to measure sophistication by the number of capabilities rather than by the usefulness of the right ones.

Economic design also concerns invocation patterns. A safe capability that is called constantly because its description is too broad may still become a cost problem. Rate limits, quotas, and better tool descriptions can therefore be understood not merely as safety or operations features, but as economic control mechanisms. This is one more reason why capability engineering sits at the intersection of software design, governance, and business judgment.

LinkMind's emphasis on reusable skills is most persuasive when read through this portfolio lens. Reuse lowers duplicate build cost, but only if the platform is disciplined about which shared assets deserve long-term care and which experiments should remain local or temporary.